Lab 7
Off-policy Actor-Critic methods: DDPG and TD3

November, 2026

1 Goal of the Lab

We have implemented, on one hand, value based algorithms like `DQN` that learn with sample efficiency but only in environments that are controlled using a set of discrete actions. On the other hand, we have implemented methods like `VPG` and `A3C` that learn not so efficiently but can deal with environments with continuous actions. Now we will implement first Deep Deterministic Policy Gradient (DDPG) [1] and and improvement over it called Twin Delayed Deep Deterministic policy gradient (TD3) [2] that are more sample efficient than on-policy gradient methods but at the same time can control continuous actions. They are described in the slides, and you can find the references in the bibliography.

2 Deep Deterministic Policy Gradient (DDPG) [4 pts]

In this exercise, we will implement the DDPG method (see figure 1). To show that it works in a continuous environment setting, we will work in the **continuous** version of the LunarLander-v3 environment that is created as follows:

```
env = gym.make("LunarLander-v3",continuous=True, render_mode='rgb_array')
```

In this environment, that you know well, actions are redefined to be 2 continuous actions, each in the range $[-1..1]$: the first one, controls the main engine and the second one controls left and right engines.

Note about parameters used for training: Use the ones described in Supplementary Information (section 7) of Experiment details in the original paper [1]. Even when the authors did not apply DDPG to the `LunarLander` problem, they seem to work also well in this problem. **Notice that DDPG is very sensitive to hyper-parameters**, architecture of the neural network and initialization of weights. I strongly warn you against trying to find the parameters by yourself instead of following the recommendations of the paper![1].

2.1 Understand the auxiliary classes:

In order to program the DDPG algorithm, I supply you with some classes implemented in different files. To implement DDPG you have to understand them.

1. **Network Architecture:** In file `ActorCriticNetworks.py` you will find implemented the Actor and Critic that will be used for DDPG. Do not change anything. In figures 2 and 3 you can see the architecture of the Actor and the Critic. Observe the following:
   (a) Use of batch normalization layers as suggested in the paper [1]
   (b) Use of custom initialization of weights, also as recommended in [1], section 7.

[1] You can try but I will not pay for your therapist afterwards...

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

Figure 1: DDPG algorithm

(c) Architecture for the Critic in which the state and action follow different paths of different length, so preprocessing of the state is done independently of the action. Finally, both paths converge to one layer where we sum the last layers of both paths.

2. **Noise classes:** DDPG is Deterministic in the sense that the policy always return the "best action", like DDQN. So, an exploration method has to be added in the method, like epsilon greedy in DQN. In DDPG exploration is implemented by adding a random value (noise) to the action. Random value can be generated using different stochastic procedures. The simplest one is sampling from a Normal distribution $N(0, \sigma)$. Another way to explore that seems to help better to DDPG for exploration is to follow the *Ornstein-Uhlenbeck* stochastic noise, that is very similar, but takes into consideration the last random value generated (see here for details). All these methods are implemented in the file `Noise.py` that defines classes for each kind of noise with a `sample` procedure that is used to choose the noise to add for exploration.

2.2 Programming Tasks:

1. **Replay Buffer:** You should implement a `ReplayBuffer` for DDPG. Remember that it is an off-policy method, so we need it! You can reuse the one we used for DQN. Notice that most of the program we will implement is very similar to DQN. That is because DDPG can be seen as an extension of DQN to continuous action spaces with the help of an Actor that is trained to find the maximum of the Q-value network.

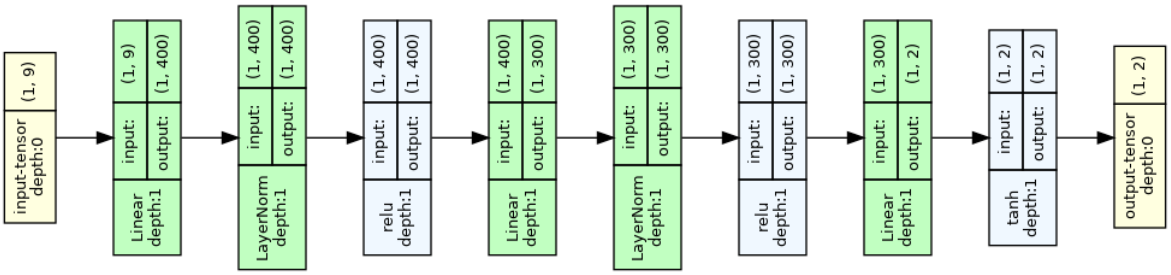


Figure 2: Actor architecture consists of 2 linear layers followed by Batch normalization and `Relu` function activation layers, and ending in a linear layer with a `tanh` activation layer that ensures that output is in range -1..1 which is the range of the actions in this continuous problem.

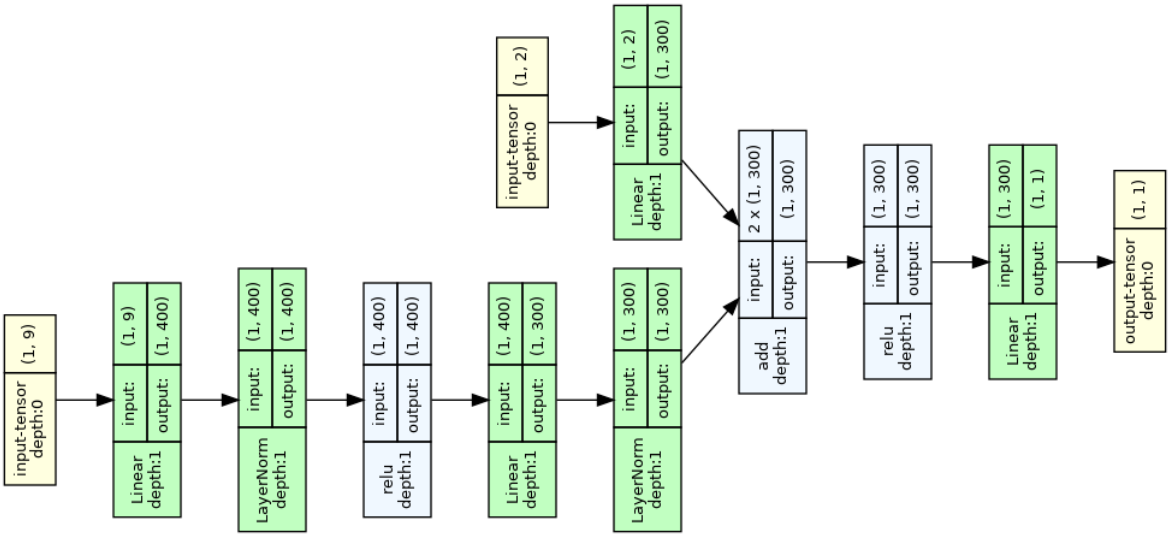


Figure 3: Critic architecture consists of 2 paths, lower one for the state and upper one for the action, that are combined with a sum of layers. The output is a single unbounded (no function activation) value - $Q(s, a)$

2. **Init of DDPG agent:** In init of the `DDPG` class in `DDPG.py` file, define the Critics and the Actors.
3. **Choose action:** In `DDPG.py` file, implement the method `choose_action` that given an state, returns the action to execute according to the Actor.
4. **Critic loss:** Implement the method `compute_critic_loss` that, given a batch of data, computes the Minimum Square Error for the Bellman equation similarly as it is done in DQN.
5. **Actor loss:** Implement the method `compute_actor_loss` that try to modify the actor to maximize the Q-values for a batch of states.
6. **Learn method: Apply gradients** In the learn method, when `ReplayBuffer` has enough data, get a batch of data and compute and update parameters of the Critic and Actor
7. **Learn method: Update targets** Do a soft update of the target neural networks with $\tau = 0.001$ as sugested in the paper.

After you have ended the programming tasks, run your program to see the performance. I reommend you to use GoogleColab with acces to a GPU to run the program because it takes some time to learn. For that I supply you with the file `RunDDPG_Colab.ipynb`. In figure 4 you can see the evolution of learning of a typical run. It takes about 1h and 1h30 to end the training in GoogleColab in a GPU node.

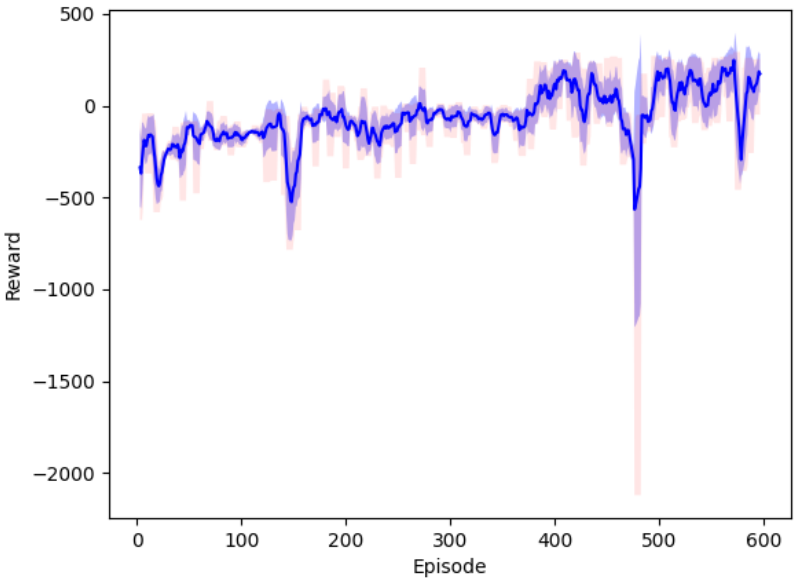


Figure 4: Illustrative learning curve. An anomalous bad trial changes the reward scale, but performance improves from approximately -400 to +200. Use the declared transition budget for the comparisons in this lab.

For the DDPG–TD3 comparison, declare a common training budget T in environment transitions before running the experiments. Use one training seed per algorithm, record all seeds and hyperparameters, and plot return against environment transitions. Do not select the best seed or stop one algorithm early because it reaches a reward threshold. Figure 4 is illustrative; reaching its final score is not a grading requirement. These two runs are an exploratory comparison, not evidence of a statistically significant performance difference. During the TD3 run, save the checkpoint required in Section 4.

3 Twin Delayed DDPG (TD3) [3 pts]

DDPG is an off-policy method able to learn with continuous action spaces. However, it is very sensible to parameters and takes some time to learn. Some improvements can be made. This is the aim of [2] that introduces three simple changes that you should implement in a copy of your solution file `DDPG.py` that you will call `TD3.py`. The changes are the following (see full algorithm in figure 5):

3.1 Programming Tasks:

1. **Target policy smoothing:** TD3 adds clipped Gaussian noise to the action used to compute the critics' target:

$$\widetilde{a}' = \mathrm{clip}\big(\mu_{\phi^-}(s') + \mathrm{clip}(\epsilon, -c, c), -1, 1\big), \qquad \epsilon \sim \mathcal{N}(0, \sigma^2_{\mathrm{targ}} I).$$

   Use $\sigma_{\mathrm{targ}} = 0.2$ and $c = 0.5$ for this action range. This noise is used inside the Bellman target; it is different from the exploration noise added to the action executed in the environment. Use Gaussian exploration noise from `Noise.py`, and keep its settings unchanged across all TD3 variants. Do not add exploration noise during evaluation.

2. **Twin critics and clipped double-Q learning:** Implement two critics Q_{θ_1} and Q_{θ_2} and their target counterparts. Train both critics towards the same target,

$$y = r + \gamma(1 - d) \min_{i=1,2} Q_{\theta_i^-}(s', \widetilde{a}'),$$

Algorithm 1 TD3

Initialize critic networks Q_{θ_1}, Q_{θ_2}, and actor network π_ϕ
with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta_1' \leftarrow \theta_1$, $\theta_2' \leftarrow \theta_2$, $\phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
for $t = 1$ **to** T **do**
  Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,
  $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'
  Store transition tuple (s, a, r, s') in $\mathcal{B}$

  Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$
  $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', \tilde{a})$
  Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
  if t mod d **then**
    Update ϕ by the deterministic policy gradient:
    $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$
    Update target networks:
    $\theta_i' \leftarrow \tau\theta_i + (1 - \tau)\theta_i'$
    $\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$
  end if
end for

Figure 5: TD3 Algorithm

where $d = 1$ for a true terminal transition. Treat a time-limit truncation separately from a true terminal state when deciding whether to bootstrap. Taking the minimum is intended to reduce overestimation; it does not guarantee an accurate or unbiased target. Use Q_{θ_1}, rather than the minimum of the two critics, for the actor objective.

3. **Delayed policy and target updates:** Update both critics at every learning iteration, but update the actor and all target networks only every second critic update (`policy_delay=2`). This gives the critics more updates between policy changes. Count learning iterations, rather than episodes or environment steps, when applying this delay. Keep the soft-update coefficient $\tau = 0.001$.

4 Understanding TD3 through controlled ablations [3 pts]

Investigate what each TD3 mechanism changes in your implementation. AI tools are allowed to orchestrate experiments, but base your discussion on measurements from your code, rather than on a generic description of TD3.

Add configuration switches so that the same TD3 code can run the following variants. Change only the indicated mechanism; keep architectures, optimizers, learning rates, exploration, batch size and critic updates unchanged.

Variant	Minimum in target	Target noise	`policy_delay`
Full TD3	yes	$\sigma_{\text{targ}} = 0.2$	2
No minimum	no: use Q_{θ_1}	$\sigma_{\text{targ}} = 0.2$	2
No smoothing	yes	zero	2
No delay	yes	$\sigma_{\text{targ}} = 0.2$	1

In the *no minimum* variant, still train both critics: only replace the minimum in the Bellman target by the first target critic. In every variant, the actor maximizes the first online critic. In the *no delay* variant, both the actor and the target networks are updated at every critic update; do not change τ or the learning rates to compensate.

After implementing the switches, run for 3-5 different seeds (runs) each combination, training the algorithms for $T = 300.000$ **steps**. Set the number of runs per configuration depending on your computation power. For me training 5 different runs for each experiment took 9 hours on my local machine. Alternatively, you can try to run them in Colab in parallel each variant using different accounts.

Another option to save some computing time is to set an episode boundary near $T/3$, after learning has started, save the full TD3 agent: online and target state dictionaries, optimizer states, replay buffer, update counter and random-generator states. After that, continue each experiment from an independent copy of this checkpoint. This save 1/3 of computation time for each execution.

Expected result is a graphic of the mean score every 1.000 steps of training for every combination. You should show the graph and also comment, according your results, the ablation study.

5 Deliver your work

Write a PDF report of at most four pages (excluding references) containing the exploratory DDPG–TD3 comparison and the controlled study in Section 4. Include your implementation, the script or notebook that generates the diagnostics. Submit these files together in a Zip file to the *racó*. Checkpoints and the full replay buffer may remain local; the submitted measurements must support every figure and table in the report. The total mark is 10 points: 4 for DDPG, 3 for TD3, and 3 for the controlled study.

References

[1] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Iclr 2016*, 2016, pp. 1–14. [Online]. Available: http://arxiv.org/abs/1509.02971

[2] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *35th International Conference on Machine Learning*, feb 2018. [Online]. Available: http://arxiv.org/abs/1802.09477